

APP DEV 1 PROJECT REPORT

Name : N Parthiv Kumar

Roll No : 23f3001787

Email : 23f3001787@ds.study.iitm.ac.in

About Me : I am a student at IIT Madras BS Degree program, doing it as a dual degree . I have made applications, games, AI/ML models & have special interest in ideating and building them.

1.Project Details -

Project Title: TrekkingMax - Adventure Management Platform

Problem Statement:

To build a web-based platform that bridges the gap between adventure seekers (trekkers) and trek organizers (staff/admins). The platform must handle role based authentication, real time slot management for trek bookings and support tracking of treks and its users.

Approach:

The application is to be built with Flask as the core backend framework. It should use a role based architecture (Admin, Staff, Trekker) to serve customized dashboards. The frontend is built using html ,css , Jinja2 templating and mainly used bootstrap5 for styling .Used SQLAlchemy for database operations and storing in a database.db file.

The flow I followed to build - was to do the raw structure of the website as I go through the milestones .Once all 6 milestones were completed I did ui styling mostly with bootstrap5 . For navigation top bar , dropdowns from profile icon, building accordions was all done using bootstrap5 . To make the ui look good I decided to use a mountain bg image and do a white - green theme for text and bg. For icons like location,warning,etc i used bootstrap icons .

When an admin creates a trek , the staff who is assigned to it must approve it and then open it on their side, setting appropriate trekker count ,managing the trekker(remove them), etc .Then it will be visible in the home page so trekker can book.

2. AI/LLM & Resources used Declaration -

- 1) AI google gemini used for bootstrap guidance(for navbar , collapse meaning , other functionalities like bootstrap icons how to use).
- 2) Flask-login how to use guidance
- 3) Guidance on small things , like how to import image , how to create relationship & foreignkey setup for models in sqlalchemy , which import for generating hash key.
- 4) Documentation :
 - > jinja : <https://jinja.palletsprojects.com/en/stable/templates/#filters>
 - > flasksqlalchemy : <https://flask-sqlalchemy.readthedocs.io/en/stable/>
 - > flask : <https://flask.palletsprojects.com/en/stable/>
 - > flask login: <https://flask-login.readthedocs.io/en/latest/>
 - > bootstrap icons : <https://icons.getbootstrap.com/?q=nav%2F>
 - > bootstrap: <https://getbootstrap.com/docs/5.3/components/modal/#how-it-works>

AI usage only for suggestions/guidance & the use of gemini is limited to 10-15% for guidance purposes only.

Note : [seed.py](#) for seeding few treks initially was made using AI.

3. Technologies and Frameworks Used -

Flask = Core backend web framework

SQLAlchemy - Object Relational Mapper for SQLite database so we dont need to write sql queries

Jinja-Template engine for rendering dynamic HTML pages (if ,else,for ,block)

Bootstrap 5 - Frontend styling

Flask-Login - User authentication, session management and route protection

Werkzeug Security - Secure password hashing

SQLite- Lightweight local database for storing user and booking data , portability

4. DATABASE SCHEMA & ER DIAGRAM -

Tables:

User > stores user profiles and roles

(id, username, email, password, role, status)

Trek > stores trek details and lifecycle

(id, name, location, difficulty, duration_days, available_slots, status, staff_id, start_date)

Booking > the intersection table logging user bookings

(id, user_id, trek_id, status, booking_date)

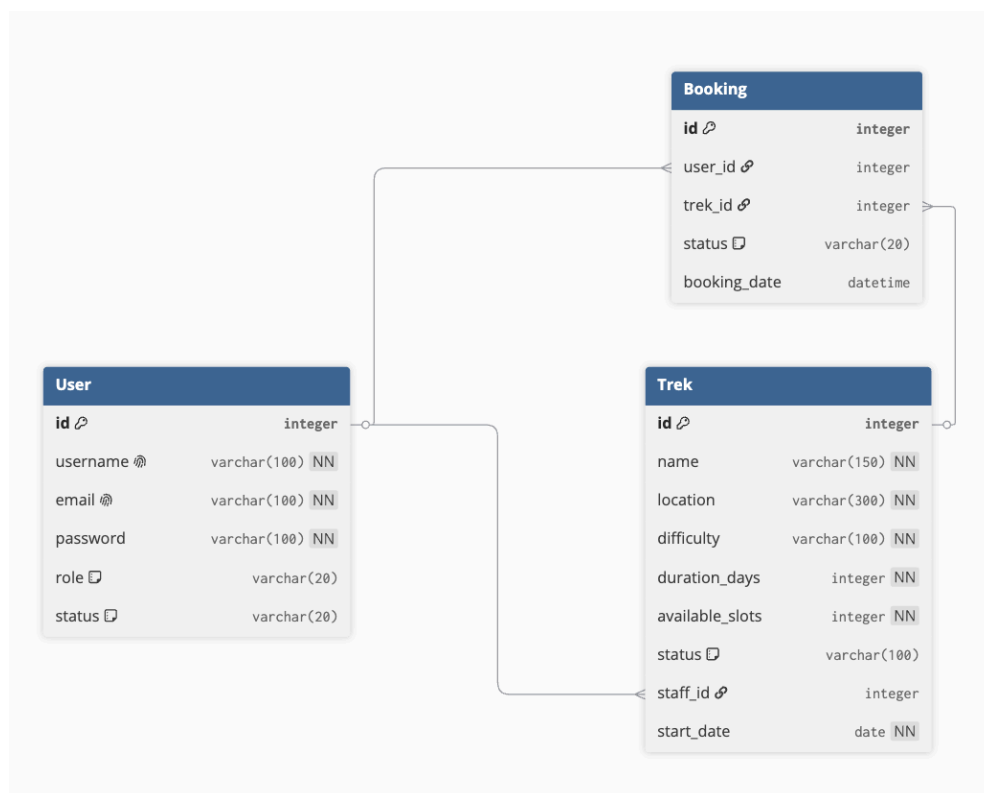
Relationships:

One-to-Many > Staff - Trek (A staff member manages multiple treks)

One-to-Many > Trekker - Booking (A trekker can have multiple bookings)

One-to-Many > Trek - Booking (A single trek has multiple bookings)

Staff, trekker, admin come as role in User itself



5. Application Routes (Core Endpoints)

As my code doesn't use any API that returns JSON-based response but rather is Server-Side Rendered Web App.

/ > GET - Homepage search engine with filtering and trek booking

/login & /register > GET, POST - for User authentication and role assignment

/book_trek/<id> >GET - Initiates the booking process and concurrency checks

/process_payment/<id> > POST - Finalizes booking and decrements available trek slots

/admin_dashboard > GET - Master control panel for user management and trek creation

/staff_dashboard > GET - Panel for staff to manage assigned treks and participants

/trekker_dashboard > GET - to see booking history , complete payment of booking , see upcoming booking

6. Architecture and Features

Architecture Overview:

- > app.py - Main Flask application entry point containing all routing and database logic.
- > /templates - Jinja2 HTML templates structured with a base inheritance model (base.html which has navbar).
- > /static - Custom CSS (style.css) with core theme chosen forest green..
- > /instance/database.db - The SQLite database file.

Implemented Features:

- > RBAC separating Admins, Staff and Trekkers.
- > Automated lifecycle management (if the day of trek happens it goes from open to ongoing.
- > logic to auto cancel incomplete bookings if the date occurs
- > search with filtering (Location, Name(on search bar), Difficulty, Duration days).
- > Global check to prevent blacklisted user
- > bootstrap for most of styling and components

7. Video Presentation

Drive link :

<https://drive.google.com/file/d/1qyuzz5DkjBDI1NwGoE3ivlaKOUgLBGvc/view?usp=sharing>